

TOKENIZAÇÃO

LISTA DE EXERCÍCIOS

- I) Quais são os três componentes fundamentais de um sistema de tokenização?
 - a) Pré-processamento, normalização e lematização.
 - b) Vocabulário (quais tokens existem), segmentação (como o texto é dividido) e mapeamento (tokens para IDs inteiros).
 - c) Embedding, atenção e decodificação.
 - d) Codificação, compressão e indexação.
- II) O que descreve o problema de **fertility** (fertilidade) na tokenização multilíngue?
 - a) A incapacidade do tokenizador de lidar com emojis e caracteres especiais.
 - b) O crescimento exponencial do vocabulário ao quebrar palavras de um mesmo idioma em tokens formados por menos caracteres.
 - c) A variação no número de tokens necessários para representar texto semanticamente equivalente em diferentes idiomas: idiomas com alta fertilidade consomem mais tokens, reduzindo a janela de contexto efetiva e aumentando o custo computacional.
 - d) A tendência do BPE de criar tokens muito longos para certos idiomas.
- III) O algoritmo BPE (Sennrich et al., 2016) constrói o vocabulário em qual direção e utiliza qual critério para selecionar os pares a mesclar?
 - a) De cima para baixo (top-down), removendo tokens com menor probabilidade.
 - b) De baixo para cima (bottom-up), mesclando iterativamente o par de símbolos adjacentes mais frequente no corpus.
 - c) De baixo para cima (bottom-up), mesclando o par que maximiza a verossimilhança do modelo de linguagem unigrama.
 - d) De cima para baixo (top-down), particionando tokens longos nos pontos de menor frequência de bigrama.
- IV) Qual tokenizador é utilizado pelo BERT e qual é o tamanho aproximado de seu vocabulário?
 - a) Byte-level BPE, com vocabulário de 50.257 tokens.
 - b) SentencePiece Unigram, com vocabulário de 32.000 tokens.
 - c) WordPiece, com vocabulário de aproximadamente 30.522 tokens.
 - d) BPE padrão, com vocabulário de 128.256 tokens.
- V) Por que o SentencePiece (Kudo & Richardson, 2018) é descrito como *independente de idioma* (language-agnostic)?
 - a) Porque suporta apenas o algoritmo BPE, que é universalmente aplicável.
 - b) Porque utiliza um vocabulário fixo de 256 bytes, igual para todos os idiomas.
 - c) Porque trata a entrada como um fluxo bruto de bytes/Unicode sem pré-tokenização por espaços em branco, representando o espaço como um caractere regular (`_`).

- d) Porque aprende um vocabulário separado para cada idioma detectado no corpus.
- VI) No **Byte-level BPE** (adotado pelo GPT-2), qual é o tamanho do vocabulário base *antes* de qualquer merge ser aplicado, e por quê esse tamanho garante que nenhum token seja desconhecido (<unk>)?
- 65.536 tokens (um por ponto de código Unicode BMP), cobrindo os caracteres mais comuns de todas as línguas.
 - 256 tokens (um por valor de byte possível), pois qualquer texto pode ser codificado em UTF-8 como uma sequência de bytes no intervalo [0,255].
 - 128 tokens (caracteres ASCII), suficientes para texto em inglês sem OOV.
 - 1.024 tokens (pares de bytes comuns), reduzindo o comprimento das sequências sem perder cobertura.
- VII) O tokenizador Unigram Language Model (Kudo, 2018) difere do BPE principalmente em qual aspecto do processo de construção do vocabulário?
- O Unigram usa merges guiados por frequência de bigramas, enquanto o BPE usa verossimilhança.
 - O Unigram é top-down: começa com um vocabulário grande (todos os substrings até certo comprimento) e itera removendo os tokens cuja ausência causa a menor queda na verossimilhança do corpus.
 - O Unigram aplica dropout nos merges durante o treinamento, enquanto o BPE usa merges determinísticos.
 - O Unigram é restrito a modelos que usam SentencePiece, enquanto o BPE pode ser usado com qualquer framework.
- VIII) O **BPE Dropout** (Provlkov et al., 2020) e a **subword regularization** do Unigram (Kudo, 2018) compartilham qual objetivo de treinamento?
- Reduzir o tamanho do vocabulário durante o treinamento para diminuir o uso de memória.
 - Aumentar a velocidade de tokenização ao pular merges pouco frequentes.
 - Servir como *data augmentation*: expor o modelo a múltiplas segmentações válidas de um mesmo texto, tornando-o mais robusto a variações na tokenização.
 - Garantir que cada token apareça no mínimo uma vez por batch de treinamento.
- IX) Considere o corpus de treinamento: low (×5), lower (×2), newest (×6), widest (×3). Aplicando o algoritmo BPE a partir de caracteres individuais, quais são os dois primeiros merges realizados (em ordem) e qual é a frequência do par em cada caso?
- Merge 1: (l, o) → lo (freq 7); Merge 2: (lo, w) → low (freq 7).
 - Merge 1: (e, s) → es (freq 9); Merge 2: (es, t) → est (freq 9).
 - Merge 1: (s, t) → st (freq 9); Merge 2: (e, s) → es (freq 9).
 - Merge 1: (n, e) → ne (freq 6); Merge 2: (ne, w) → new (freq 6).
- X) Usando o mesmo corpus (low×5, lower×2, newest×6, widest×3), o WordPiece seleciona merges maximizando o score:

$$\text{score}(x, y) = \frac{\text{freq}(xy)}{\text{freq}(x) \cdot \text{freq}(y)}.$$

Calcule os scores para os pares (e, s) e (i, d). Qual par o WordPiece mescla primeiro? Esse resultado é igual ao BPE?

- a) (e, s): score $\approx 0,059$; (i, d): score $\approx 0,333$. WordPiece mescla (i, d) primeiro, pois esse par coocorre quase exclusivamente junto; o BPE teria mesclado (e, s) por ter frequência bruta maior (9 vs 3).
- b) (e, s): score $\approx 0,529$; (i, d): score $\approx 0,111$. WordPiece mescla (e, s) primeiro, concordando com o BPE.
- c) (e, s): score = 9; (i, d): score = 3. Ambos os algoritmos sempre concordam na escolha do primeiro merge.
- d) (e, s): score $\approx 0,059$; (i, d): score $\approx 0,333$. WordPiece mescla (e, s) primeiro porque frequência bruta é o critério de desempate.

SOLUÇÕES

I — Resposta: (b)

Um sistema de tokenização é definido por três componentes:

1. **Vocabulário:** o conjunto de tokens reconhecidos pelo modelo — quais sequências de caracteres são unidades atômicas.
2. **Segmentação:** o algoritmo que divide um texto de entrada em uma sequência de tokens do vocabulário.
3. **Mapeamento:** a função que converte cada token em um ID inteiro (e vice-versa), necessária para alimentar o modelo com tensores numéricos.

Essas três decisões em conjunto determinam silenciosamente o desempenho do modelo, a eficiência computacional e a equidade multilíngue.

II — Resposta: (c)

Fertility (fertilidade) mede quantos tokens um tokenizador produz para representar texto em um determinado idioma. Idiomas com alta fertilidade requerem mais tokens para transmitir a mesma informação semântica que o inglês requer com menos tokens. Por exemplo, a frase “Hello, how are you?” pode gerar 6 tokens em inglês, mas o equivalente em japonês pode exigir 14 ou mais tokens com o mesmo tokenizador. Consequências práticas:

- Mais tokens = mais FLOPs por inferência.
- Janela de contexto efetiva menor para idiomas de alta fertilidade.
- Modelos treinados majoritariamente em inglês têm fertilidade desfavorável para outros idiomas, criando desigualdade de desempenho.

III — Resposta: (b)

O BPE (Byte Pair Encoding) é um algoritmo **bottom-up** e **guloso**:

1. Inicia com um vocabulário de símbolos individuais (caracteres ou bytes).
2. Conta a frequência de todos os pares de símbolos adjacentes no corpus.
3. Mescla o par mais frequente em um novo símbolo composto.
4. Repete o processo por k iterações (hiperparâmetro que define o tamanho do vocabulário).

A tabela de merges aprendida é aplicada deterministicamente na inferência, na mesma ordem em que foi aprendida. O BPE foi proposto para tradução automática neural por Sennrich et al. (2016) e tornou-se a base dos tokenizadores da família GPT.

IV — Resposta: (c)

O BERT (Devlin et al., 2019) usa **WordPiece** com um vocabulário de ≈ 30.522 tokens. O WordPiece foi desenvolvido originalmente para sistemas de reconhecimento de voz (Schuster & Nakajima, 2012) e posteriormente adaptado para modelos de linguagem (Wu et al., 2016). Diferencia-se do BPE pelo critério de merge (verossimilhança em vez de frequência) e pelo uso do prefixo **##** para marcar continuações de palavras (ex.: “playing” $\rightarrow$ “play”, “##ing”).

V — Resposta: (c)

A maioria dos tokenizadores depende de regras de pré-tokenização baseadas em espaços em branco e pontuação — regras que funcionam bem para o inglês mas são inadequadas para idiomas como japonês, chinês ou tailandês, que não usam espaços para delimitar palavras. O SentencePiece elimina essa dependência ao tratar o texto de entrada como um fluxo bruto de bytes ou pontos de código Unicode, sem nenhuma segmentação prévia. O espaço em branco é substituído pelo símbolo **_** e tratado como qualquer outro caractere, tornando a tokenização completamente reversível e uniforme em todos os idiomas. O SentencePiece serve como *framework*: internamente pode usar BPE ou o modelo Unigram como algoritmo de segmentação.

VI — Resposta: (b)

O UTF-8 codifica qualquer ponto de código Unicode em uma sequência de 1 a 4 bytes, onde cada byte assume um valor no intervalo $[0, 255]$. Ao operar sobre bytes brutos, o byte-level BPE começa com exatamente **256 tokens base** — um para cada valor de byte possível. Consequência direta: qualquer texto, em qualquer idioma ou sistema de escrita, incluindo emojis e caracteres especiais, pode ser representado sem nenhum token desconhecido (**<unk>**). Os merges do BPE são então aplicados sobre esses 256 bytes para construir tokens maiores. O GPT-2 usa esse esquema com um vocabulário final de 50.257 tokens (256 bytes + 50.000 merges + 1 token especial (**<E0S>**)), e o GPT-3/4 expande para 100.256 tokens.

VII — Resposta: (b)

Enquanto o BPE é **bottom-up** (começa pequeno e cresce mesclando), o Unigram é **top-down** (começa grande e encolhe podando):

1. Inicializa o vocabulário com todos os substrings do corpus até um comprimento máximo (podendo ter centenas de milhares de entradas).
2. Define uma distribuição de probabilidade unigrama sobre o vocabulário.
3. Para cada token candidato à remoção, calcula a queda na verossimilhança do corpus se ele fosse eliminado.
4. Remove os tokens com menor queda (os menos úteis) — tipicamente 10–20% do vocabulário por iteração.
5. Repete até atingir o tamanho de vocabulário desejado.

Uma propriedade exclusiva do Unigram é poder gerar *múltiplas* segmentações válidas de um mesmo texto com probabilidades associadas, algo explorado pela subword regularization como augmentation de dados.

VIII — Resposta: (c)

Ambas as técnicas introduzem **aleatoriedade controlada** na segmentação durante o treinamento:

- **BPE Dropout** (Provilkov et al., 2020): durante o treinamento, cada merge do BPE é ignorado com probabilidade p (tipicamente $p = 0,1$). A mesma palavra pode ser tokenizada de maneiras diferentes em diferentes passos, como `low est_`, `lo w est_` ou `l o w est_`.
- **Subword regularization** (Kudo, 2018): usa o modelo probabilístico do Unigram para amostrar uma segmentação dentre as válidas, em vez de sempre escolher a de maior probabilidade.

O efeito é equivalente ao dropout nas camadas de rede: força o modelo a ser robusto a múltiplas representações de uma mesma sequência, melhorando a generalização — especialmente em cenários com dados escassos ou idiomas de alta morfologia.

IX — Resposta: (b)

Estado inicial — cada palavra representada como caracteres individuais com marcador de fim de palavra (`_`):

- `l o w _` (×5), `l o w e r _` (×2)
- `n e w e s t _` (×6), `w i d e s t _` (×3)

Frequência dos pares adjacentes relevantes:

- (e, s) : aparece em “newest” (×6) e “widest” (×3) $\Rightarrow \text{freq} = 9$
- (s, t) : aparece em “newest” (×6) e “widest” (×3) $\Rightarrow \text{freq} = 9$
- (l, o) : aparece em “low” (×5) e “lower” (×2) $\Rightarrow \text{freq} = 7$

Quando há empate na frequência, os pares costumam ser ordenados lexicograficamente ou pela ordem de aparição. O par (e, s) é mesclado antes de (s, t) pois é anterior a s na ordem lexicográfica.

Merge 1: $(e, s) \rightarrow es$ (freq 9). Vocabulário passa a incluir `es`.

Estado após merge 1:

- `n e w e s t _` (×6), `w i d e s t _` (×3)

Merge 2: o par mais frequente agora é (es, t) com $\text{freq} = 9$. $(es, t) \rightarrow est$. Vocabulário passa a incluir `est`.

X — Resposta: (a)

Frequências individuais dos caracteres (calculadas somando sobre todas as ocorrências no corpus):

- $\text{freq}(e) = 1 \times 2 + 2 \times 6 + 1 \times 3 = 2 + 12 + 3 = 17$ (“lower”: 1 e ; “newest”: 2 e ’s; “widest”: 1 e)
- $\text{freq}(s) = 6 + 3 = 9$

- $\text{freq}(i) = 3$ (só em “widest”)
- $\text{freq}(d) = 3$ (só em “widest”)

Frequências dos pares:

- $\text{freq}(es) = 6 + 3 = 9$
- $\text{freq}(id) = 3$

Scores WordPiece:

$$\text{score}(e, s) = \frac{\text{freq}(es)}{\text{freq}(e) \cdot \text{freq}(s)} = \frac{9}{17 \times 9} = \frac{9}{153} \approx 0,059.$$

$$\text{score}(i, d) = \frac{\text{freq}(id)}{\text{freq}(i) \cdot \text{freq}(d)} = \frac{3}{3 \times 3} = \frac{3}{9} \approx 0,333.$$

Conclusão: o WordPiece mescla (i, d) primeiro ($\text{score} \approx 0,333$), embora (e, s) tenha frequência bruta três vezes maior (9 vs 3).

Intuição: i e d coocorrem quase exclusivamente juntos no corpus — toda vez que i aparece, d é o próximo símbolo. O score captura essa alta informação mútua. Por outro lado, e e s aparecem frequentemente de forma *independente* em outros contextos, reduzindo o score mesmo com alta frequência de coocorrência absoluta. O BPE ignora essa distinção e escolheria (e, s) simplesmente por ter maior contagem.